MQTT test: NodeMCU → broker → laptop

The NodeMCU publishes its own stats every 2 seconds to topic `test/01`; a Python script on the laptop receives and prints them.

Files in this folder

`nodemcu_mqtt_pub/nodemcu_mqtt_pub.ino`	Publisher. Arduino sketch (C++) for the NodeMCU.
`sub.py`	Subscriber. Python script for the laptop.

How it fits together

`NodeMCU (publisher)` $\xrightarrow{\text{Wi-Fi, MQTT}}$ `Broker (Mosquitto)` $\xrightarrow{\text{MQTT}}$ `sub.py (subscriber)`

The broker is the "post office": the NodeMCU posts messages to topic `test/01`, and every program subscribed to `test/01` gets a copy. The sender (C++) and the receiver (Python) don't need the same language: they both speak the MQTT protocol.

Choose your setup first

The sketch has a **broker block**: keep **exactly one** `BROKER` line uncommented. `sub.py` takes the broker as an argument instead (no editing), and must point to the **same broker** as the sketch.

	Option A: broker on your laptop (default)	**Option B: public broker**
In the sketch	`BROKER = "<laptop IP>"`	`BROKER = "broker.hivemq.com"`
Run `sub.py`	`python sub.py` (= localhost)	`python sub.py broker.hivemq.com`
Needs internet?	No. Traffic stays on the local Wi-Fi.	Yes.
Works on college Wi-Fi?	Yes (tested)	**No**: the college network blocks MQTT to the internet
Works on phone hotspot?	Yes	Yes
Needs Mosquitto installed?	Yes, on the laptop	No
Private?	Yes	No: anyone can read topic `test/01`

What about the proxy? The proxy (host `172.31.2.3`, port `8080`) is only for reaching the internet, and it **cannot carry MQTT**: it only allows HTTPS-style connections on port 443, and the MQTT ports are refused (tested). So there are no proxy lines in the code:

- Option A never needs the proxy (local traffic).
- Option B needs a **phone hotspot with the laptop's proxy turned OFF**.
- The proxy is only needed in **Arduino IDE** on the college network, to download the library in step 1 (**File** → **Preferences** → **Network** → **Manual proxy**, host `172.31.2.3`, port `8080`, no username/password).

Step 1: Install the MQTT library in Arduino (once)

1. Arduino IDE → **Tools** → **Manage Libraries...**
2. Search **PubSubClient**.
3. Install **"PubSubClient by Nick O'Leary"**.

On the laptop, Python needs `paho-mqtt`: `pip install paho-mqtt` (once).

Step 2: Open and edit the sketch

1. **File** → **Open** → `nodemcu_mqtt_pub/nodemcu_mqtt_pub.ino`
2. Edit the lines at the top:

```
const char* WIFI_SSID = "IIITA";          // Wi-Fi name
const char* WIFI_PASS = "";               // Wi-Fi password ("" if none)
const char* TOPIC     = "test/01";
const char* BROKER = "172.29.0.43";       // OPTION A: laptop IP
// const char* BROKER = "broker.hivemq.com"; // OPTION B: public
```

3. **Option A:** find the laptop's IP with `ipconfig` (Windows) → *Wireless LAN adapter Wi-Fi* → *IPv4 Address*. It can change from day to day, so check it again if the NodeMCU suddenly cannot connect.
4. The Wi-Fi must be **2.4 GHz** (ESP8266 cannot see 5 GHz). On a phone hotspot choose the 2.4 GHz band / "Maximize Compatibility".

Step 3: Upload

1. Plug in the NodeMCU, select board **NodeMCU 1.0 (ESP-12E Module)** and its **COM port**.
2. Close the Serial Monitor if open, then click **Upload**.

Step 4: Watch the NodeMCU

Tools → **Serial Monitor**, **9600 baud**, press **RST**. Good output:

```
Connecting to Wi-Fi 'IIITA'.....
Wi-Fi OK, my IP: 172.29.x.x
Connecting to MQTT broker 172.29.0.43 ... OK
Publishing to topic: test/01
sent  {"uptime_s":3,"rssi_dbm":-55,"free_heap":50112,"a0":4}
```

Step 5: Receive on the laptop

Open PowerShell in this folder and run:

```
cd C:\path\to\mqtt_test
python sub.py                    # broker = localhost (this laptop)
python sub.py 172.29.1.135       # broker = another machine, e.g. the Pi
python sub.py broker.hivemq.com  # public broker (hotspot only)
```

Use the one that matches `BROKER` in the sketch. Every 2 seconds:

```
13:04:13  test/01  {"uptime_s":63,"rssi_dbm":-89,"free_heap":50112,"a0":4}
```

Stop with **Ctrl+C**.

What the values mean

Field	Meaning
`uptime_s`	Seconds since the NodeMCU last started. Goes up by 2 each message. Drops back to 0 if the board restarts.
`rssi_dbm`	Wi-Fi signal strength in dBm. Always negative; **closer to 0 = stronger**. About −50 excellent, −70 good, −85 weak, −90 barely working. Move the board closer to the router/hotspot and watch it rise.
`free_heap`	**Free RAM in bytes.** The ESP8266 has about 80 KB of RAM; around 50,000 bytes free is normal for this sketch. If it stays constant (it should), there is no memory leak. If it slowly falls, the program is leaking memory.
`a0`	Reading of the analog pin A0, from 0 to 1023 (0 V to about 3.3 V). Nothing is connected, so it shows small random noise (about 0–10). Touch the pin with a finger and it jumps. A real analog sensor (e.g. a light sensor) would go here.

These are placeholder "sensor" values, because the ESP8266 has **no built-in temperature sensor**. Later a DHT22 replaces them with real temperature and humidity.

Troubleshooting

Symptom (Serial Monitor)	Cause / fix
Stuck on `Connecting to Wi-Fi.....`	Wrong name/password, network is 5 GHz, or it needs a browser/username login (ESP8266 can't do that). Use a phone hotspot.
`failed (state -2)` repeating	Can't reach the broker. Option A: wrong laptop IP, Mosquitto not running, or the network blocks device-to-device traffic. Option B: you are on the college network; use a hotspot.
Gibberish	Serial Monitor baud must be 9600 (same as `Serial.begin(9600)` in the sketch).
`sub.py` connects but prints nothing	Different broker or topic than the sketch. Both files must use the same option and `test/01`.
`sub.py`: `ConnectionRefusedError`	Option A: Mosquitto is not running on the laptop.

Mosquitto on Windows (option A), one-time setup: in `C:\Program Files\mosquitto\mosquitto.conf` add `listener 1883 0.0.0.0` and `allow_anonymous true`, restart the Mosquitto service, and allow TCP port 1883 in Windows Firewall. Without this, Mosquitto only accepts connections from the laptop itself.

Result (tested)

Option A on the college Wi-Fi. Left: NodeMCU sketch and its Serial Monitor (`sent {...}`). Right: `python sub.py` on the laptop receiving the same messages on `test/01` every 2 seconds.

